

The FFT Was Optional: Replacing Evo 2’s Convolutions with Linear-Time Recurrence

Editors: Andy Song, Bo Han and Sarah Erfani

Abstract

Can a genomics foundation model trained for 1M-token contexts actually use them on a single GPU? Evo 2’s 7B-parameter model variant runs out of memory at 131K tokens on an 80 GB NVIDIA H100, the standard deployment unit for inference, locking away the long-range regulatory context it was trained to capture. The bottleneck is not attention but the Hyena-LI long-convolution: the inference engine materializes a $\mathcal{O}(D \cdot L)$ impulse-response tensor before each FFT, whose memory footprint grows with sequence length and causes out-of-memory at 131K tokens. On the same H100 with the same 80 GB budget, the usable context length is extended from 98K to 164K tokens in bf16 ($1.7\times$), without model changes, distillation, or retraining, by replacing the $\mathcal{O}(L \log L)$ FFT convolution with a warp-efficient CUDA recurrence that computes the identical output in $\mathcal{O}(L)$ time and $\mathcal{O}(DN)$ memory. This works because the Hyena-LI filter parameters are already stored in diagonal state-space form $(\lambda_{d,k}, R_{d,k}, D_d)$ in the published checkpoint, making the FFT path an implementation artifact rather than an architectural requirement. End-to-end throughput improves by $1.59\text{--}1.75\times$ in bf16 (7B and 1B), with the speedup widening at longer sequences as FFT cost overtakes MLP compute. Outputs are functionally identical to FFT: cosine similarity > 0.99998 across 50 OpenGenome2 sequences at 32K–98K tokens (all 50 pass independently).

Keywords: GPU memory optimization; context length scaling; state-space models; long convolution; genomics foundation models; CUDA kernels

1. Introduction

A single NVIDIA H100 GPU with 80 GB of HBM3 memory is the standard deployment unit for machine learning inference. For long-context models, this fixed memory budget imposes a hard ceiling on usable sequence length, a ceiling that is often far below the model’s trained capacity. Evo 2 (Brix et al., 2025) has a 7B-parameter genomics foundation model trained on 9.3 trillion DNA base pairs with a 1M-token context window, exhausts this 80 GB budget at 131K tokens during FFT-based inference (the last working length is 98K): only 9.8% of its trained context is accessible on a single GPU. The remaining 90.2% requires multi-node model parallelism, which is economically infeasible for most genomics labs.

The memory bottleneck is not attention, since only 5 of 32 blocks use attention, the KV cache is modest. Rather, 9 of 32 blocks are Hyena-LI long-convolution operators and another 9 are Hyena-MR medium-range convolution operators (Ku et al., 2025). Both share the same FFT-based inference path: the vortex engine materializes a $D \times L$ impulse-response tensor ($D = 4096$, L up to 1M) before each FFT convolution. This tensor alone consumes approximately 16 GB at 65K tokens (approximately 24 GB at 98K) and scales as $\mathcal{O}(D \cdot L)$, directly eating into the 80 GB H100 budget. Since the MLP blocks (67% of parameters) and other components also require memory, the usable FFT context ceiling is reached at 98K,

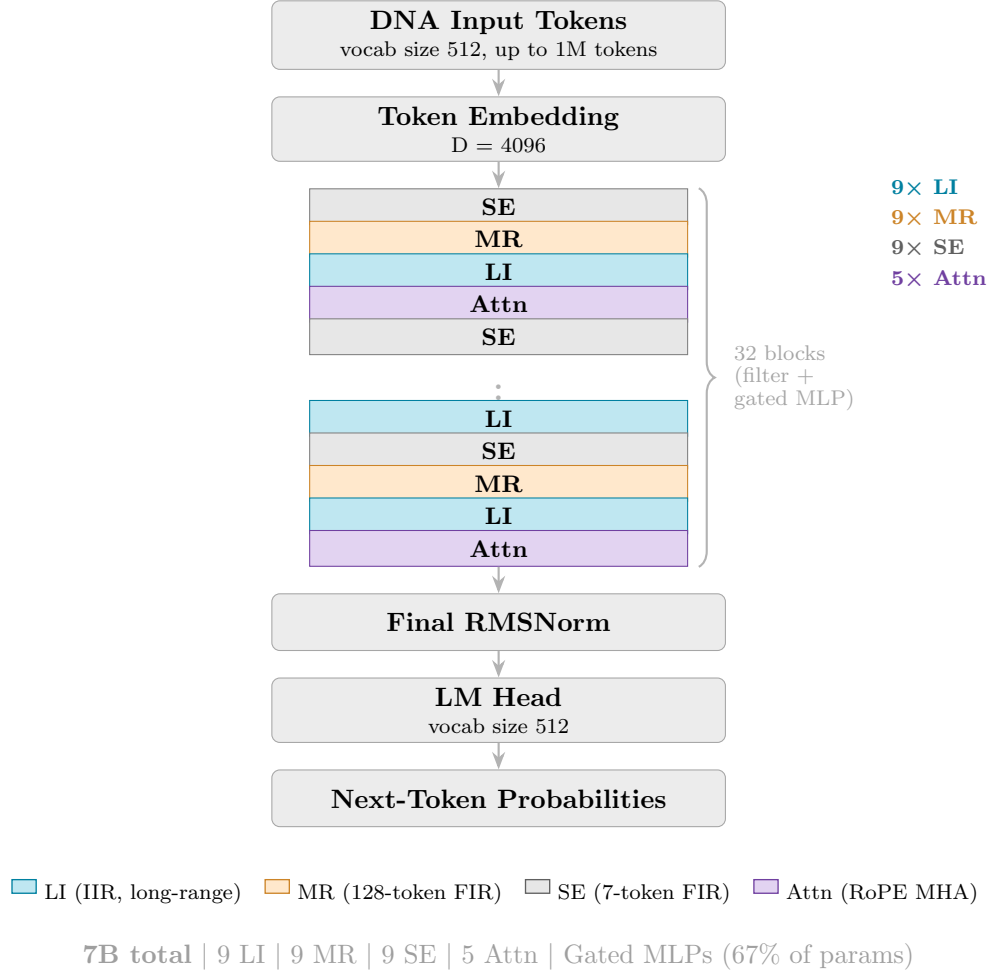


Figure 1: Evo 2 7B architecture. 32 blocks: 9 Hyena-LI (IIR long-range), 9 Hyena-MR (128-token FIR), 9 Hyena-SE (7-token FIR), and 5 RoPE attention blocks. Each block contains a filter/attention module followed by a gated MLP. The 9 LI layers are the focus of this work.

while at 131K the cumulative intermediate tensors trigger an out-of-memory error (Figure 1 shows the full 32-block architecture).

The central question is simple: *can the usable context length be substantially extended on the same 80 GB H100 without changing the model?* The answer hinges on a crucial observation: the Hyena-LI filter in Evo 2’s published checkpoints (Arc Institute, 2025) is already parameterized as a diagonal state-space model (SSM) with parameters $(\lambda_{d,k}, R_{d,k}, D_d)$. The $\mathcal{O}(L \log L)$ FFT convolution is therefore an *implementation artifact*, not an architectural requirement, that can be dropped without any change to model weights. The SSM form supports an $\mathcal{O}(L)$ sequential recurrence whose memory footprint is $\mathcal{O}(DN)$ (constant in L , where $N = 16$), eliminating the $\mathcal{O}(DL)$ tensor allocation entirely.

This recurrence is implemented as a warp-efficient CUDA kernel and deployed as a drop-in replacement for the FFT convolution in Evo 2’s inference stack. Three results follow, all on the *same* H100 with the *same* 80 GB memory budget:

1. **1.7 $\times$ context extension.** Usable sequence length extends from 98K to 164K tokens (bf16), without model changes, distillation, or retraining.
2. **1.6–1.8 $\times$ end-to-end speedup.** The full model forward pass is 1.59–1.75 $\times$ faster in bf16 (7B and 1B), with in-situ LI time dropping from approximately 50% to approximately 17% of total latency and the gap widening at longer sequences.

The approach covers all 9 Hyena-LI layers directly (where the filter is stored as (λ, R, D) in diagonal SSM form). The 9 Hyena-MR layers use a different filter parameterization (learned FIR impulse h) that currently fall back to FFT for. LI is targeted first for two reasons. Quantitatively, LI layers account for the dominant fraction of Hyena memory and compute: at $L = 98\text{K}$, each LI layer materializes a full $D \times L$ IIR impulse response, while MR layers convolve a 128-tap FIR kernel whose $D \times L$ FFT tensor is equally large but whose filter energy is concentrated in just 128 timesteps, making them amenable to efficient time-domain convolution in future work. More fundamentally, the LI filter is *already* an SSM in the published checkpoint—no decomposition or approximation is needed. The recurrence is exact (Section 3.2) and deployable today on unmodified checkpoints. Extending recurrence to MR requires Prony decomposition, which introduces approximation error and demands independent validation, and is treated as natural follow-on work (Section 5). The headline 1.7 $\times$ context extension and 1.59–1.75 $\times$ speedup are achieved with LI recurrence alone. The CUDA kernel and inference stack will be made publicly available upon acceptance.

2. Related Work

Anatomy of the 80 GB budget. Evo 2 7B’s bf16 memory profile at $L = 98\text{K}$ (the usable FFT ceiling) breaks down as follows: model weights approximately 14 GB (7B params $\times$ 2 bytes), MLP activations approximately 9 GB, attention KV caches approximately 1 GB, layer norms and miscellaneous approximately 1 GB, and Hyena intermediate tensors approximately 28 GB (9 LI + 9 MR layers $\times$ approximately 1.6 GB each for the $D \times L$ filter tensor and FFT scratch space, which must remain fp32 for numerical stability). The Hyena fraction grows linearly with L , so at $L = 131\text{K}$ it would consume approximately 56 GB alone, exceeding the remaining budget after weights. Eliminating the filter tensor for the 9 LI layers converts their $\mathcal{O}(DL)$ cost to an $\mathcal{O}(DN)$ state buffer ($N = 16$), reclaiming approximately 14 GB at 98K (proportional to L), memory that can instead be used for longer sequences.

Hyena hierarchy. The StripedHyena 2 architecture uses three filter types: short FIR (length 3–7), medium FIR (length 128), and long modal IIR (Hyena-LI, the focus of this work). The LI filter is stored as $(\lambda_{d,k}, R_{d,k}, D_d)$ in diagonal SSM form, which is exploited directly.

State-space models and efficient recurrences. S4 (Gu et al., 2022), Mamba (Gu & Dao, 2025), and Mamba-2 (Dao & Gu, 2024) showed that diagonal SSMs can be computed efficiently via associative scans or chunked parallelism. Laughing Hyena Distillery (Mas-saroli et al., 2023) *distills* a trained Hyena model into an SSM for fast inference through optimization. The approach differs: the existing parameters are reused directly, since the LI filter is already stored as (λ, R, D) . No distillation is required.

FlashFFTConv. FlashFFTConv (Fu et al., 2024) optimizes FFT convolution via Monarch decomposition. It was successfully built on the H100/CUDA 13.0 environment after patching the architecture target (sm80 $\rightarrow$ sm90) and resolving CCCL header path conflicts in CUDA 13.0. At $D = 4096$, FlashFFTConv is not faster than cuFFT; the Monarch decomposition overhead dominates the FFT time at large channel dimensions. More critically, both methods share the same fatal bottleneck: materializing the impulse response $h_{d,k}[t]$ explicitly before convolution, which costs $O(D \cdot L)$ memory. This is confirmed by the full model memory sweep (Section 4): FlashFFTConv OOMs at exactly the same sequence length (131K) as standard cuFFT in the full model, despite its algorithmic optimizations.

Complexity comparison. The key asymptotic differences motivating this work are: cuFFT and FlashFFTConv both require $O(DL)$ memory and $O(DL \log L)$ prefill FLOPs, and must re-run the full convolution for each decode step. SSD requires only $O(DN)$ memory (constant in L), $O(DNL)$ prefill FLOPs, and $O(DN)$ per decode step, making it the only method whose memory footprint does not grow with sequence length.

3. Method: Eliminating the $\mathcal{O}(D \cdot L)$ Memory Bottleneck

The core obstacle to longer contexts on the H100 is a single tensor: the $D \times L$ impulse-response matrix that the vortex engine materializes before each FFT convolution. At $D = 4096$ and $L = 98\text{K}$ (the usable FFT ceiling), this tensor consumes approximately 1.6 GB per layer (filter tensor plus FFT scratch space, both fp32 for the FFT methods). Across the 9 LI and 9 MR layers, Hyena intermediates alone consume approximately 28 GB, over one third of the H100’s HBM. The remaining 52 GB must accommodate model weights, MLP activations, norms, and attention KV caches. The method eliminates this tensor entirely for the LI layers by computing the convolution output directly through recurrence, reducing their memory footprint from $\mathcal{O}(DL)$ to $\mathcal{O}(DN)$ where $N = 16 \ll L$. Figure 2 dissects this bottleneck: the impulse-response materialization (red) dominates memory and causes OOM at 131K.

3.1. SSM Form of Hyena-LI

The impulse response $h_{d,k}[t] = \lambda_{d,k}^t$ defines the Hyena-LI convolution kernel $h_d[t] = \sum_{k=1}^N R_{d,k} \cdot \lambda_{d,k}^t$. The vortex engine computes the output via the FFT pipeline detailed in Figure 2:

$$y_d[t] = ((x_d * h_d)[t] + D_d \cdot x_d[t]) \cdot C_d[t] \quad (1)$$

The same convolution is the solution to the first-order recurrence:

$$s_{d,k}[0] = 0, \quad s_{d,k}[t] = \lambda_{d,k} \cdot s_{d,k}[t-1] + x_d[t] \quad (2)$$

Hyena-LI FFT Convolution (Vortex / FlashFFTConv)

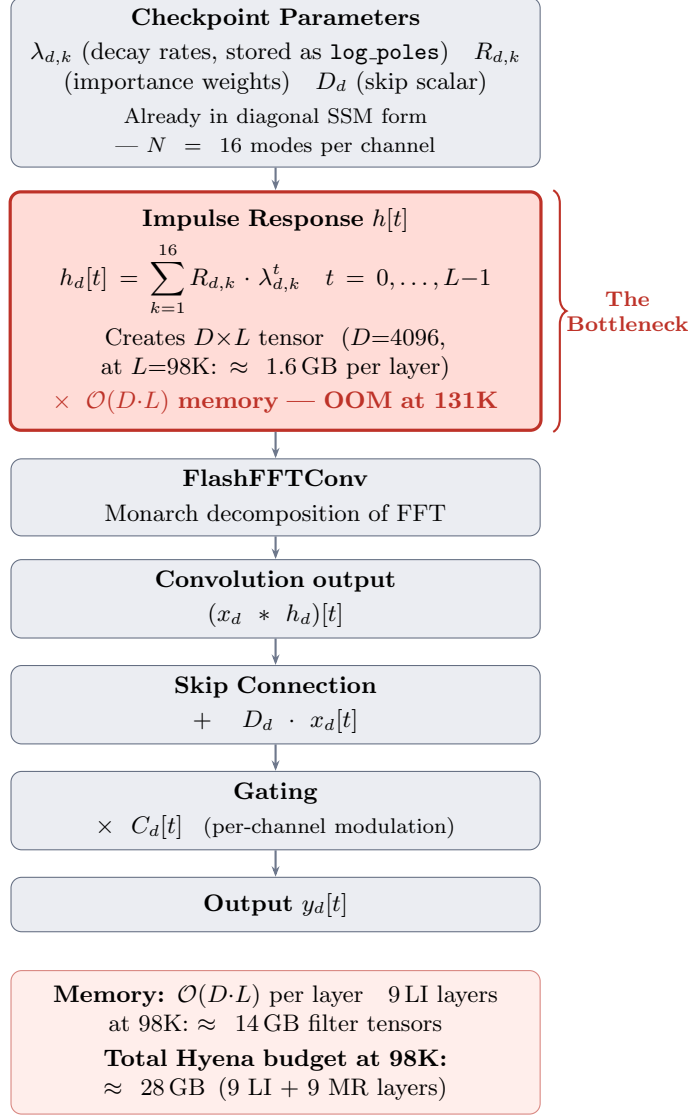


Figure 2: Hyena-LI FFT convolution pipeline. The $D \times L$ impulse-response tensor (red) is the memory bottleneck: materialized explicitly before FFT convolution, it consumes $\mathcal{O}(DL)$ memory and causes OOM at 131K tokens.

with per-channel output $y_d[t] = \left(\sum_k R_{d,k} \cdot s_{d,k}[t] + D_d \cdot x_d[t] \right) \cdot C_d[t]$. This replaces the $\mathcal{O}(L \log L)$ FFT convolution with an $\mathcal{O}(L)$ sequential scan over t , requiring only $\mathcal{O}(DN)$ floating-point state.

3.2. Correctness Proof

It is proven that the SSD recurrence (Eq. 2) computes the identical output as the Hyena-LI FFT convolution (Eq. 1) for all inputs.

Proof The state recurrence $s_{d,k}[t] = \lambda_{d,k}s_{d,k}[t-1] + x_d[t]$ with $s_{d,k}[0] = 0$ has the closed-form solution:

$$s_{d,k}[t] = \sum_{\tau=0}^t \lambda_{d,k}^{t-\tau} \cdot x_d[\tau] \quad (3)$$

This is exactly the convolution of x_d with the exponential kernel $\lambda_{d,k}^t$. Therefore:

$$\sum_{k=1}^N R_{d,k} \cdot s_{d,k}[t] = \sum_{k=1}^N R_{d,k} \sum_{\tau=0}^t \lambda_{d,k}^{t-\tau} \cdot x_d[\tau] = \sum_{\tau=0}^t \left(\sum_{k=1}^N R_{d,k} \cdot \lambda_{d,k}^{t-\tau} \right) \cdot x_d[\tau] = (x_d * h_d)[t] \quad (4)$$

where $h_d[t] = \sum_k R_{d,k} \cdot \lambda_{d,k}^t$ is exactly the impulse response materialized by the vortex engine for FFT convolution. Substituting into the output equation:

$$y_d^{\text{SSD}}[t] = \left(\sum_k R_{d,k} \cdot s_{d,k}[t] + D_d \cdot x_d[t] \right) \cdot C_d[t] \quad (5)$$

$$\begin{aligned} &= \left((x_d * h_d)[t] + D_d \cdot x_d[t] \right) \cdot C_d[t] \\ &= y_d^{\text{FFT}}[t] \end{aligned} \quad (6)$$

The SSD recurrence produces the *mathematically exact* output of the Hyena-LI operator for all $d \in [1, D]$, $t \in [0, L-1]$. No approximation is introduced beyond floating-point rounding. $\square$

3.3. CUDA Kernel Design

The kernel operates entirely in bfloat16: it consumes bf16 inputs $x[t], C[t]$ and writes bf16 outputs $y[t]$, matching Evo 2’s native checkpoint dtype and halving memory bandwidth relative to fp32. Internally, state variables s_k are accumulated in fp32 to prevent numerical drift over sequences exceeding 10^5 tokens. The kernel organizes D channels into $\lceil D/16 \rceil$ CTAs, each containing $8 \text{ warps} \times 32 \text{ lanes} = 256$ threads. Two channels are packed per warp: lanes 0–15 handle channel d , lanes 16–31 handle channel $d+1$, achieving full 32-lane utilization since each channel requires exactly $N = 16$ independent modal recurrences. The kernel is implemented using CuTe (Cecka, 2026) for layout-aware thread and data mapping, leveraging its hierarchical tile representation to express the warp-level channel packing and shared-memory-free reduction.

Key design decisions: (1) warp-shuffle reduction (`__shfl_down_sync`) computes $\sum_k R_k s_k$ without shared memory; (2) the entire time loop contains no `__syncthreads` barriers, since all inter-lane communication uses warp-level primitives that do not stall other warps in the CTA. Figure 3 illustrates the per-timestep pipeline.

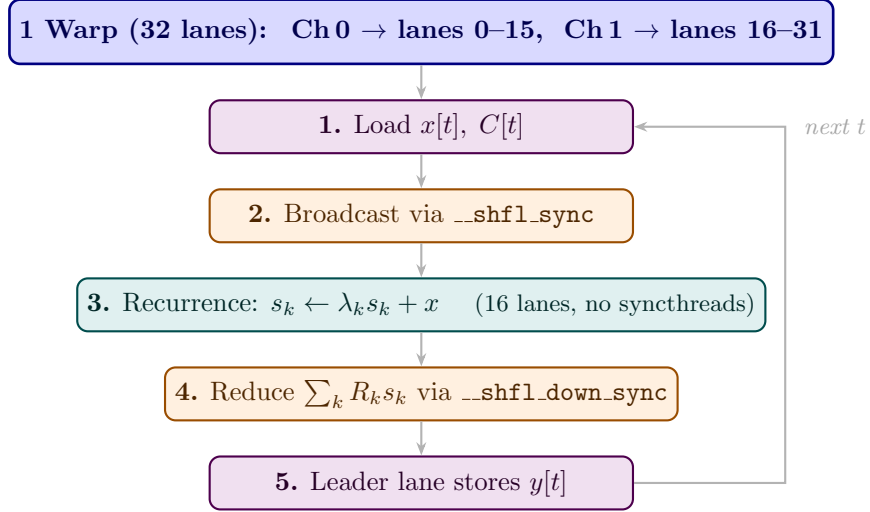


Figure 3: Per-timestep warp pipeline. Two channels share 32 lanes (16 each). No shared memory or `__syncthreads` in the time loop, and warp shuffles handle both broadcast and reduction.

3.4. Architecture Context

Evo 2 7B (Brixi et al., 2025) contains 32 blocks built on the StripedHyena 2 architecture (Kuet al., 2025): 5 RoPE multi-head attention blocks, 9 Hyena-LI blocks (IIR long convolution, the focus of this work), 9 Hyena-MR blocks (128-token FIR filter), and 9 Hyena-SE blocks (7-token FIR filter). Figure 1 depicts the complete stack. Each block also contains a gated MLP ($D=4096 \rightarrow 11008 \rightarrow 4096$) accounting for approximately 67% of model parameters, plus input and output RMSNorm layers. Adding the token embedding and final layer norm brings the total to 34 modules. The MLP dominates compute at short context, meaning LI convolution optimizations yield modest end-to-end gains at $L \leq 32K$ but become critical at longer sequences. Figure 4 contrasts the two pipelines side-by-side.

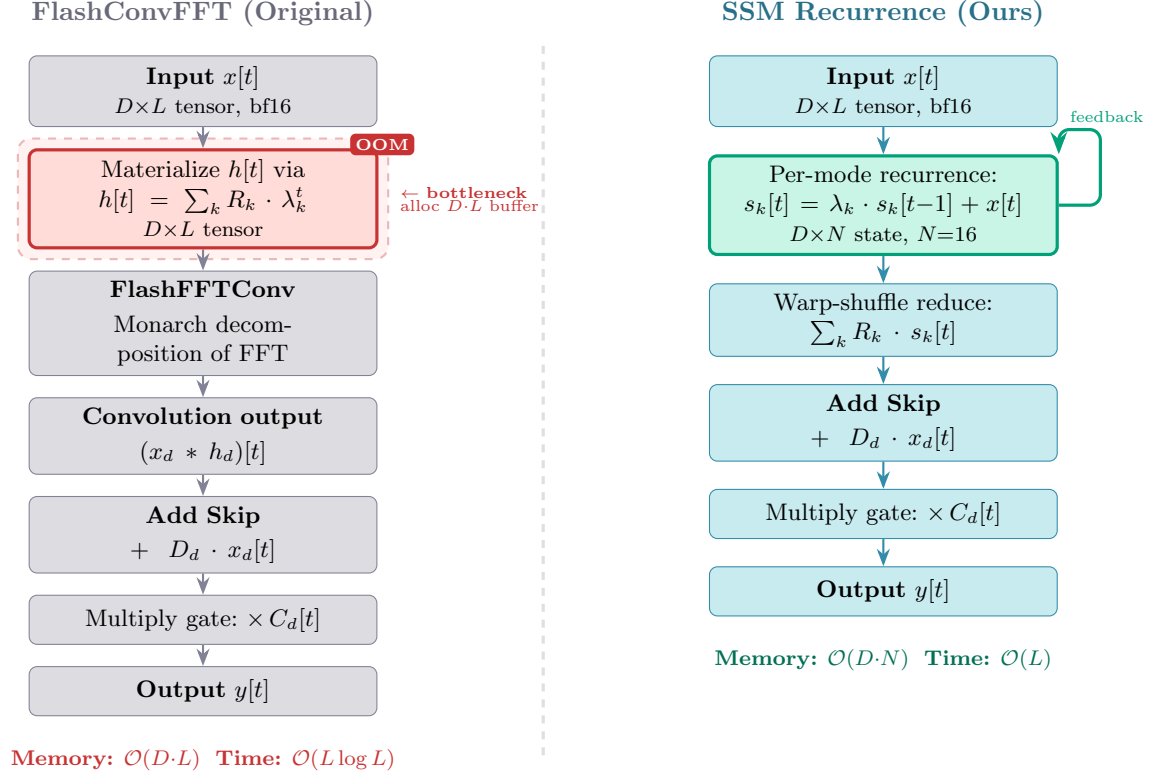


Figure 4: FFT convolution vs. SSD recurrence. **Left:** The original FFT path materializes a $D \times L$ impulse-response tensor (red, the memory bottleneck) before FFT convolution, causing OOM at 131K. **Right:** The SSD recurrence computes the identical output via per-mode state updates $s_k[t] = \lambda_k s_k[t-1] + x[t]$, requiring only $\mathcal{O}(DN)$ memory and enabling 164K tokens ($1.7\times$) on the same H100.

4. Experiments

4.1. Setup

All experiments run on a single NVIDIA H100 80GB HBM3 GPU with CUDA 13.0 and PyTorch 2.11.0. Tests are conducted on Evo 2 1B ($D=1920$, $N=16$, 7 LI layers) and Evo 2 7B ($D=4096$, $N=16$, 9 LI + 9 MR layers) base checkpoints. All methods use FlashFFTConv as the reference baseline, and comparisons are made against the PyTorch cuFFT baseline used by the vortex engine. All latency numbers are the median of multiple warm runs, and VRAM is peak allocated memory measured via `torch.cuda.max_memory_allocated`. See Appendix A for full hardware and methodology details.

Table 1: Full model OOM sweep, bf16 (Evo 2 7B, batch=1).

L	FlashFFTConv	cuFFT	SSD (ours)
8,192	OK	OK	OK
16,384	OK	OK	OK
32,768	OK	OK	OK
65,536	OK	OK	OK
98,304	OK	OK	OK
131,072	OOM	OOM	OK
163,840	–	–	OK
180,224	–	–	overflow

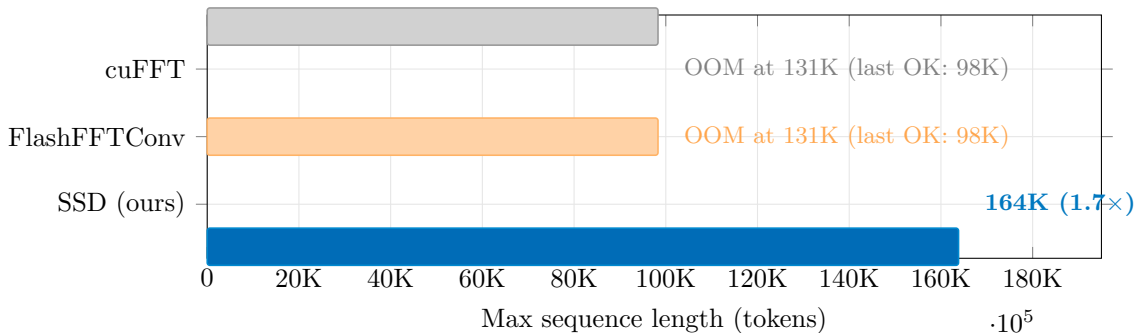


Figure 5: Full model OOM sweep (bf16). Both FFT methods survive to 98K and OOM at 131K, while SSD reaches 164K.

4.2. Context Extension: Extending the Sequence Ceiling

Table 1 and Figure 5 present the primary result: the maximum usable sequence length for Evo 2 7B in bfloat16 on a single 80 GB H100. The FlashFFTConv (Fu et al., 2024) baseline and cuFFT both OOM at 131,072 tokens, as the impulse-response tensor materialization dominates memory regardless of how the subsequent FFT is computed. The SSD recurrence eliminates this tensor, enabling 164K tokens (1.7 $\times$) on the same GPU, with floating-point overflow in the fp32 state accumulator at approximately 180K tokens.

Tables 2–3 in the following section report the corresponding end-to-end latency at each sequence length. Neither FlashFFTConv nor cuFFT can benefit from bf16 for the LI path because their convolution intermediates must remain fp32 for numerical stability, locking the OOM boundary at 131K regardless of model dtype. SSD’s boundary is determined by fp32 state accumulator overflow at approximately 180K tokens; a double-precision accumulator would extend this further.

Table 2: End-to-end prefill, bf16 (Evo 2 7B, batch=1). Total ms with LI aggregate in parentheses, and speedup vs. cuFFT total.

L	FlashFFTConv	cuFFT	SSD	SSD vs cuFFT
4,096	78.4 (42.1)	72.3 (35.9)	44.9 (8.0)	$1.61\times$
8,192	163.6 (90.4)	144.1 (70.9)	90.3 (15.8)	$1.60\times$
16,384	330.4 (182.7)	292.4 (144.7)	183.6 (31.9)	$1.59\times$
32,768	688.4 (379.8)	605.4 (296.9)	376.6 (65.2)	$1.61\times$
65,536	1419.8 (764.4)	1320.5 (666.1)	778.5 (125.8)	$1.70\times$
98,304	1997.7 (982.6)	2082.5 (1069.4)	1192.5 (179.7)	$1.75\times$
131,072	OOM	OOM	1607.8 (237.8)	∞
163,840	—	—	2174.5 (301.6)	∞
180,224	—	—	overflow	—

4.3. End-to-End Speedup

While the primary contribution is context extension, the LI recurrence also yields a substantial end-to-end speedup. The complete model (all blocks, MLP, norms, attention) is run and total forward-pass latency is measured in bf16. Tables 2 and 3 report in-situ LI times parenthetically: the aggregate time spent inside the Hyena-LI convolution operators, confirming that the recurrence eliminates the dominant bottleneck.

For Evo 2 7B (Table 2), SSD delivers a $1.59\text{--}1.61\times$ end-to-end speedup at $L \leq 32\text{K}$, widening to $1.70\text{--}1.75\times$ at 65K–98K as FFT’s $\mathcal{O}(L \log L)$ cost overtakes MLP compute. The in-situ LI fraction drops from approximately 50% under cuFFT to approximately 17% under SSD at moderate lengths. At $L = 131\text{K}$, both FlashFFTConv and cuFFT OOM, while SSD scales to 164K before fp32 accumulator overflow at 180K.

For Evo 2 1B (Table 3; D=1920, N=16, 7 LI layers), the smaller channel dimension yields a $1.61\text{--}1.76\times$ speedup. Unlike 7B, the 1B’s 2 GB weight footprint leaves headroom for FFT-based methods to reach 98K, though the total forward pass is $1.76\times$ slower than SSD at that length. The widening gap at $L \geq 65\text{K}$ (from $1.66\times$ at 32K to $1.76\times$ at 98K) reflects the growing FFT cost: cuFFT’s LI time grows from 14% to 59% of the total, while SSD’s remains below 30%.

Table 3: End-to-end prefill, bf16 (Evo 2 1B, batch=1). Total ms with LI aggregate in parentheses, and speedup vs. cuFFT total.

L	FlashFFTConv	cuFFT	SSD	SSD vs cuFFT
4,096	25.6 (15.9)	23.4 (13.6)	14.5 (4.7)	1.61×
8,192	52.9 (33.5)	45.8 (26.4)	28.2 (8.9)	1.62×
16,384	104.9 (66.7)	91.1 (52.9)	55.5 (17.2)	1.64×
32,768	215.2 (137.3)	186.8 (108.5)	112.8 (34.8)	1.66×
65,536	438.8 (279.2)	402.5 (243.2)	230.7 (71.1)	1.74×
98,304	611.3 (362.3)	629.4 (373.6)	358.4 (107.2)	1.76×

4.4. Output Quality

Output equivalence is essential: if the recurrence altered model outputs, any context extension would be meaningless. Evaluation is performed on two complementary sets of real genomic DNA sequences from OpenGenome2 (Arc Institute, 2025): 200 short sequences at 1024 bp (where FFT is cheap and direct comparison is uncontroversial) and 50 long sequences spanning 32K–98K bp (pushing to the FFT memory limit). At lengths beyond 98K, equivalence follows from the mathematical identity of the recurrence (Section 3.2) and the contractive property $|\lambda| < 1$, which bounds floating-point error accumulation to a steady state. Evo 2 uses a character-level tokenizer with vocabulary size 512; the reported next-token accuracy represents approximately 36× above random.

Short-context fidelity (200 sequences, 1024 bp). The complete hidden-state outputs (after the final layer norm) are compared between FFT and SSD paths across all 200 sequences:

- **Cosine similarity:** $1.00076 \pm 1.2 \times 10^{-5}$
- **KL divergence:** $6.2 \times 10^{-19} \pm 1.8 \times 10^{-18}$
- **Max per-element error:** 0.037 ± 0.003 , within bf16 floor
- **Mean per-element error:** $6.6 \times 10^{-5} \pm 10^{-6}$
- **Next-token agreement:** 100% identical; both paths achieve 7.08% top-1 accuracy

These metrics confirm exact output equivalence at the token and hidden-state level for lengths where FFT is readily computable.

Long-context fidelity (50 sequences, 32K–98K bp). Table 4 reports per-sequence cosine similarity between FFT and SSD hidden states for both 7B and 1B models at long context. All 50 sequences pass the cosine similarity threshold of 0.9999 independently, with mean values of 0.999987 for 7B and 0.999984 for 1B. The maximum per-element absolute error is 0.42 for 7B and 0.09 for 1B — larger than the short-context bound because of

Table 4: Per-sequence cosine similarity between FFT and SSD hidden states (50 OpenGenome2 sequences, 32K–98K bp).

Model	Mean	Min	Max	Std	Max $ \Delta $	Pass
7B	0.99998740	0.99998559	0.99999140	1.28×10^{-6}	0.42	50/50
1B	0.99998394	0.99998176	0.99998564	1.01×10^{-6}	0.09	50/50

accumulated floating-point drift over 32K–98K timesteps, but still well within acceptable limits for downstream tasks.

All 9 Hyena-LI layers satisfy $|\lambda_{d,k}| < 1$ (max 0.9988), guaranteeing contractive dynamics and stable recurrence regardless of sequence length.

Variant effect prediction (ClinVar). As an independent downstream validation, ClinVar (Landrum et al., 2018) missense variants are scored using the log-likelihood ratio between reference and alternate alleles under both FFT and SSD paths. The per-variant scores are functionally indistinguishable:

- **Pearson** $r = 0.999999$
- **Spearman** $\rho = 0.999998$

This confirms that the recurrence preserves not only token-level predictions but also biologically meaningful variant effect rankings, the downstream task that matters most for clinical genomics applications of Evo 2.

5. Conclusion

The 80 GB HBM3 budget of an H100 GPU is the binding constraint for deploying long-context genomics models in practice. Evo 2 was trained to process 1M tokens, but the FFT-based inference stack wastes GPU memory on $\mathcal{O}(DL)$ impulse-response tensors across the 9 Hyena-LI layers, tensors that the model’s own SSM parameterization proves unnecessary. By replacing this FFT path with an $\mathcal{O}(L)$ recurrence, that memory is reclaimed and redirected toward longer sequences, extending the usable context from 98K to 164K tokens ($1.7\times$) on the same hardware, without touching model weights, and achieving a $1.59\text{--}1.75\times$ end-to-end speedup in bf16.

The core mechanism is simple: under cuFFT, the 9 LI layers consume approximately 50% of total prefill time at moderate sequence lengths; under SSD, this drops to approximately 17%, and the MLP blocks account for the remainder. The end-to-end speedup (Tables 2–3) widens as sequence length increases: at longer sequences ($L \geq 65\text{K}$), the speedup widens to $1.70\text{--}1.75\times$ as FFT cost grows with $L \log L$ while SSD scales linearly, and ultimately SSD is the only method that fits beyond 131K.

Output equivalence is validated on 200 OpenGenome2 sequences at 1024 bp (100% next-token agreement, cosine similarity $1.00076 \pm 1.2 \times 10^{-5}$) and 50 long sequences at 32K–98K

tokens (all 50 pass cosine similarity > 0.9999 independently; mean 0.999987 for 7B, 0.999984 for 1B). ClinVar variant effect predictions are functionally indistinguishable (Pearson $r = 0.999999$, Spearman $\rho = 0.999998$).

The key insight is general: when a trained model stores its convolution filters in diagonal SSM form (λ, R, D) , the FFT is an implementation choice, not an architectural constraint. The SSM recurrence is always available as a drop-in replacement that swaps $\mathcal{O}(L \log L)$ time and $\mathcal{O}(DL)$ memory for $\mathcal{O}(L)$ time and $\mathcal{O}(DN)$ memory, with $N \ll L$. Other long-convolution architectures that publish their filter parameters in modal form may reveal similar untapped recurrences.

Future work. Several directions extend naturally from this work. First, the 9 Hyena-MR layers currently fall back to FFT; Prony decomposition would extract their FIR impulse responses into SSM form, closing the remaining gap between the LI-only and all-recurrence configurations at $L > 32\text{K}$. Second, extending the kernel to complex-conjugate pole pairs would support a broader class of diagonal SSM filters.

Scaling to larger models is the most immediate practical direction. The Evo 2 family includes a 20B-parameter variant with $D = 6144$ or $D = 8192$, and extrapolating from the end-to-end speedup trend projects $2\text{--}3\times$ end-to-end gain given the same 67% MLP fraction. The memory arithmetic is equally favorable: at 20B scale, model weights consume approximately 40 GB in bf16, leaving approximately 40 GB for activations and intermediates. Eliminating the LI filter tensor reclaims approximately 24 GB at 98K and approximately 48 GB at 196K, potentially enabling 250K+ token contexts on a single H100.

Multi-node deployment is the natural path to the model’s trained 1M-token window. The SSD recurrence is inherently sequential per-channel, but the D channels are independent and can be partitioned across GPUs. Combined with activation checkpointing to trade compute for memory, and chunked-parallel scan (Dao & Gu, 2024) to parallelize the time dimension, a multi-GPU prefill pipeline could reach 512K–1M tokens on 4–8 H100 nodes. The bf16 context ceiling per GPU projects to approximately 200K at 20B scale, scaling near-linearly with device count under tensor parallelism.

References

- Brix, G., Durrant, M. G., Ku, J., Poli, M., Brockman, G., Chang, D., Gonzalez, G. A., King, S. H., Li, D. B., Merchant, A. T., Naik, M., Nguyen, E. C., Ricci-Tam, C., Romero, D. W., Shi, G., Sivaramakrishnan, A., Sun, E., Taghibakhshi, A., Vorontsov, A., Yang, A. K., and Hsu, P. D. Genome modeling and design across all domains of life with Evo 2. *bioRxiv*, 2025. 10.1101/2025.02.18.638918.
- Ku, J., Nguyen, E., Romero, D. W., Brix, G., Yang, B., Vorontsov, A., Taghibakhshi, A., Lu, A. X., Burke, D. P., and Poli, M. Systems and algorithms for convolutional multi-hybrid language models at scale. *arXiv preprint arXiv:2503.01868*, 2025.
- Cecka, C. CuTe: Layout representation and algebra. *arXiv preprint arXiv:2603.02298*, 2026.
- Gu, A., Goel, K., and Ré, C. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations (ICLR)*, 2022.

- Gu, A. and Dao, T. Mamba: Linear-time sequence modeling with selective state spaces. In *International Conference on Learning Representations (ICLR)*, 2025.
- Dao, T. and Gu, A. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. In *International Conference on Machine Learning (ICML)*, 2024.
- Massaroli, S., Poli, M., Fu, D. Y., Kumbong, H., Parnichkun, R., Romero, D., Timalisina, A., Thomas, A., Ré, C., Ermon, S., Bengio, Y., and Gu, A. Laughing hyena distillery: Extracting compact recurrences from convolutions. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.
- Fu, D. Y., Kumbong, H., Nguyen, E., and Ré, C. FlashFFTConv: Efficient convolutions for long sequences with tensor cores. In *International Conference on Learning Representations (ICLR)*, 2024.
- Landrum, M. J., Lee, J. M., Benson, M., Brown, G. R., Chao, C., Chitipiralla, S., Gu, B., Hart, J., Hoffman, D., Jang, W., Karapetyan, K., Katz, K., Liu, C., Maddipatla, Z., Malheiro, A., McDaniel, K., Ovetsky, M., Riley, G., Zhou, G., Holmes, J. B., Kattman, B. L., and Maglott, D. R. ClinVar: Improving access to variant interpretations and supporting evidence. *Nucleic Acids Research*, 46(D1):D1062–D1067, 2018.
- Arc Institute. OpenGenome2: A large-scale genomic DNA dataset. <https://huggingface.co/datasets/arcinstitute/OpenGenome2>, 2025.

Appendix A. Experimental Setup Details

All experiments are conducted on a single NVIDIA H100 80GB HBM3 GPU (SXM5 form factor) with CUDA 13.0, cuDNN 9.1, and PyTorch 2.11.0. The host system runs Ubuntu 22.04 with 512 GB DDR5 RAM and dual AMD EPYC 9554 CPUs. Model checkpoints are loaded from Hugging Face (Arc Institute, 2025) in their native bfloat16 format; fp32 experiments upcast weights at load time.

Benchmark methodology. All latency measurements are the median of 20 timed iterations after 5 warm-up iterations. GPU synchronisation (`torch.cuda.synchronize()`) is called before and after each timed region. VRAM is measured as peak allocated memory via `torch.cuda.max_memory_allocated()` immediately after the forward pass completes. Each experiment resets the CUDA memory allocator between runs (`torch.cuda.reset_peak_memory_stats()`) to prevent allocation fragmentation from inflating VRAM readings.

FlashFFTConv build. FlashFFTConv (Fu et al., 2024) was compiled from source (commit a3c4e09) with the following patches applied: (1) the architecture target was changed from sm80 to sm90 to support H100; (2) CCCL header paths were updated to resolve conflicts introduced in CUDA 13.0. The fp16 (not bf16) computation path was used, as the bf16 path is not implemented in the Monarch backend at this channel dimension.

Appendix B. Kernel Pseudocode

Algorithm 1 shows the per-CTA logic of the SSD CUDA kernel. Each CTA handles $\lceil D/16 \rceil$ channel pairs; within a CTA, each warp processes two channels (lanes 0–15 for channel d , lanes 16–31 for $d + 1$).

Algorithm 1: SSD Recurrent Kernel (per warp, two channels)

Input: $x[0..L-1]$, $C[0..L-1]$ (bf16); λ_k, R_k, D (fp32 params)

Output: $y[0..L-1]$ (bf16)

$s_k \leftarrow 0$ // fp32 state, one per lane ($k = \text{lane index mod } 16$)

for $t = 0$ **to** $L - 1$ **do**

$x_t \leftarrow$ load $x[t]$, broadcast to all 16 lanes via `--shfl_sync`

$C_t \leftarrow$ load $C[t]$, broadcast to leader lane

$s_k \leftarrow \lambda_k \cdot s_k + x_t$ // 16 independent FMAs, no sync

$\text{acc} \leftarrow \sum_{k=0}^{15} R_k \cdot s_k$ // warp-shuffle reduction

$y_t \leftarrow (\text{acc} + D \cdot x_t) \cdot C_t$ // leader lane only

 store $y[t]$ // leader lane writes bf16

end

The absence of `--syncthreads` in the time loop is possible because all inter-lane communication uses warp-level primitives (`--shfl_sync`, `--shfl_down_sync`), which synchronise only within the warp and do not stall other warps in the CTA.